

Architecture & Secure Design



Secure Programming

Michael Lehmann

IT SECURITY

The software architect translates requirements into technical design

Based on the purpose of the system and the business requirements the software architect's tasks are

- ▶ High-level design of a system
- ▶ Software structure, break down into blocks of software
- ▶ Define interfaces (internal & external)
- ▶ Take decisions on technical choices
- ▶ Guidelines & principles

Basic terms and patterns

More terms

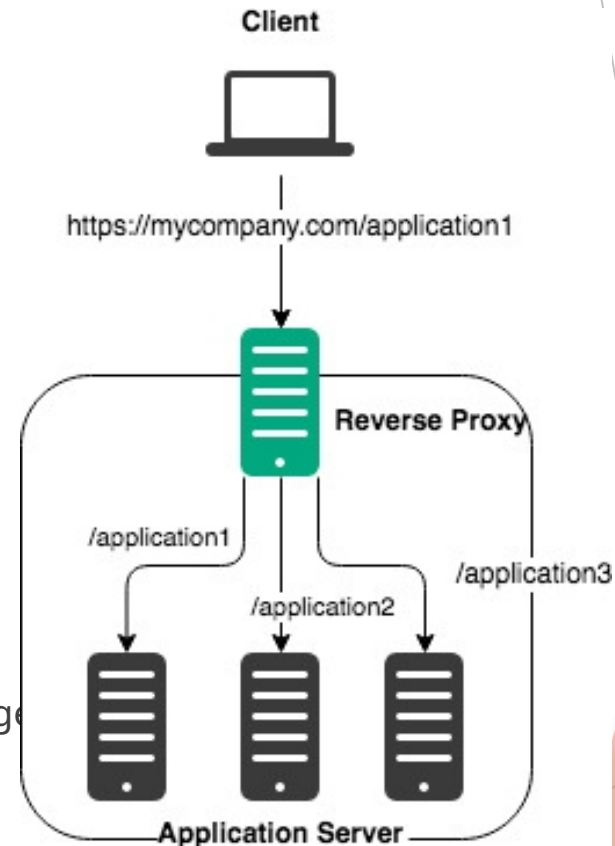
- ▶ OSI Model
 - ▶ Application: HTTP, FTP
 - ▶ Presentation: “data translator” for the network, e.g. serialisation
 - ▶ Session: “connection management”
 - ▶ Transport: TCP, UDP
 - ▶ Network: IP, ICMP
 - ▶ Data Link: Ethernet, MAC
 - ▶ Physical: Wireless, Fibre
- ▶ HTTP
 - ▶ Method, Header, Body
- ▶ Forward / Reverse Proxy

Reverse Proxy

- ▶ Entry point to an internal network

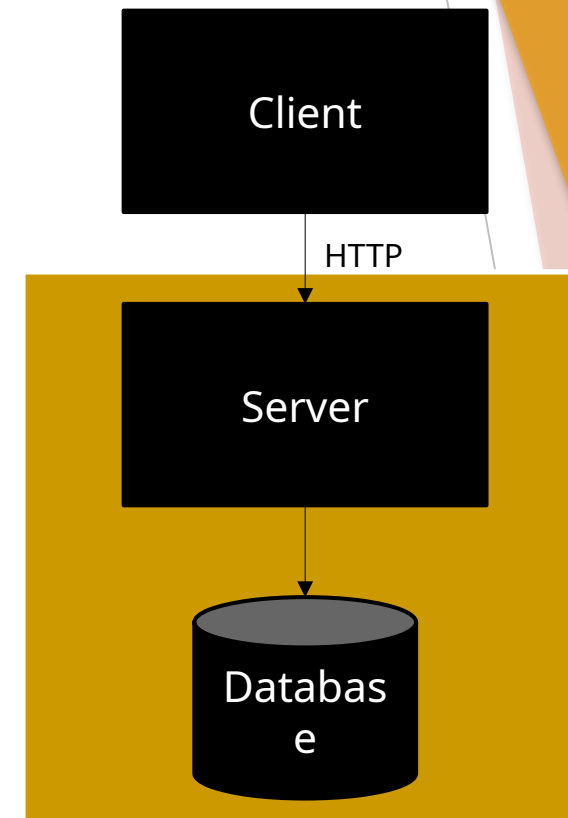
Main functionalities are

- ▶ Web Application Firewall (**WAF**), e.g. Input Validation
- ▶ Load balancing
- ▶ **Single-Sign-On** & Session Management
- ▶ Request dispatching
- ▶ HTTP Request/Response body parsing
 - ▶ HTTPS everywhere (replacing http:// with https://)
 - ▶ Rewriting: e.g. obfuscation of email addresses in HTML pages
 - ▶ Inserting analytics tags
- ▶ etc.

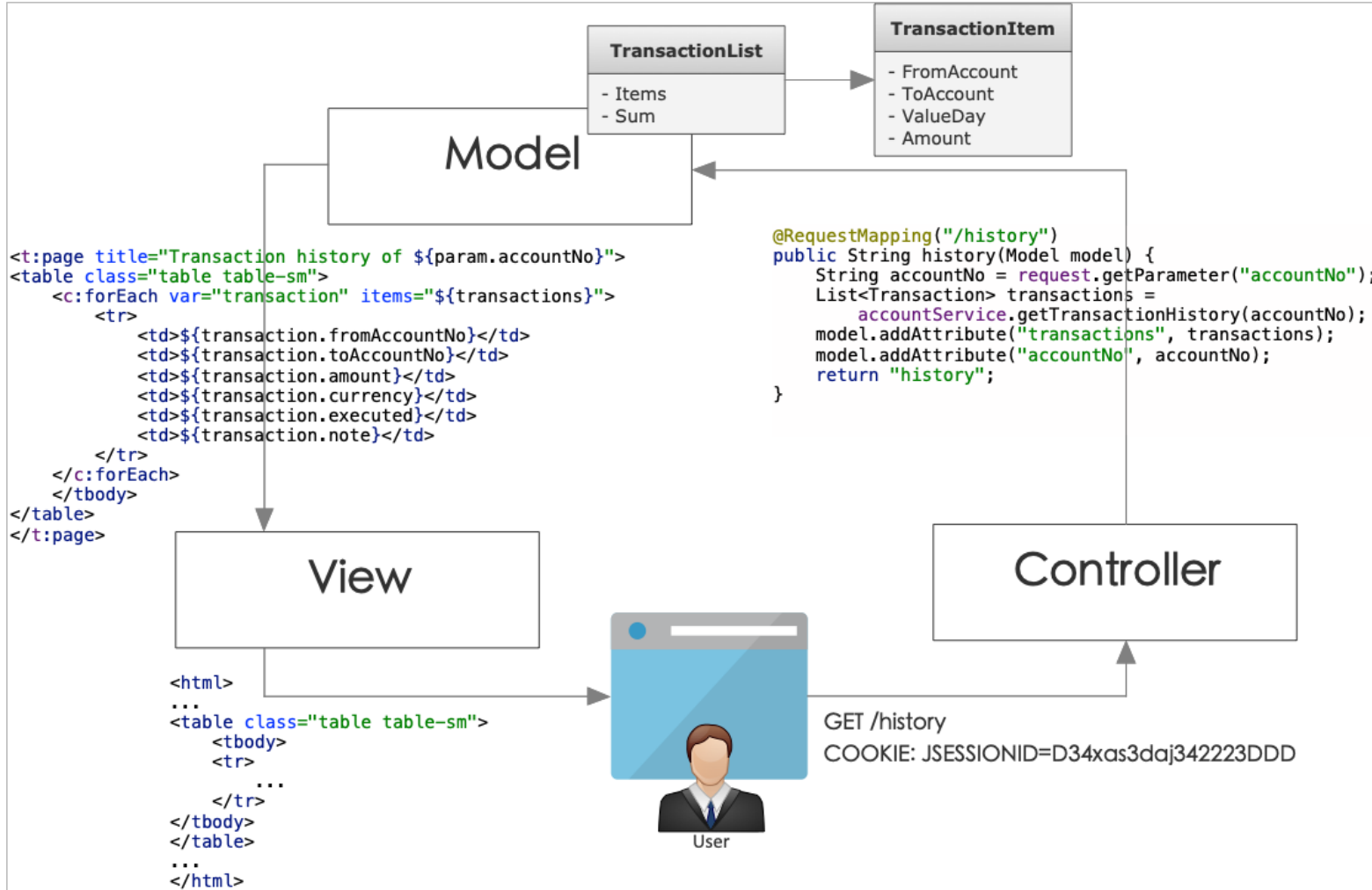


Architecture Web Application

System	Technologies	Platform
Frontend (User's device)	• HTML • CSS • JavaScript • iOS App, Android App	• Browser • User Agent • Hardware Client • Mobile App
Backend	• Java, JavaEE • Python • ASP.NET • PHP	• Web Application Server • Internal Network • OS: Linux, Windows Server
Persistence	• SQL • PL/SQL • NoSQL	• DB Server • Internal Network

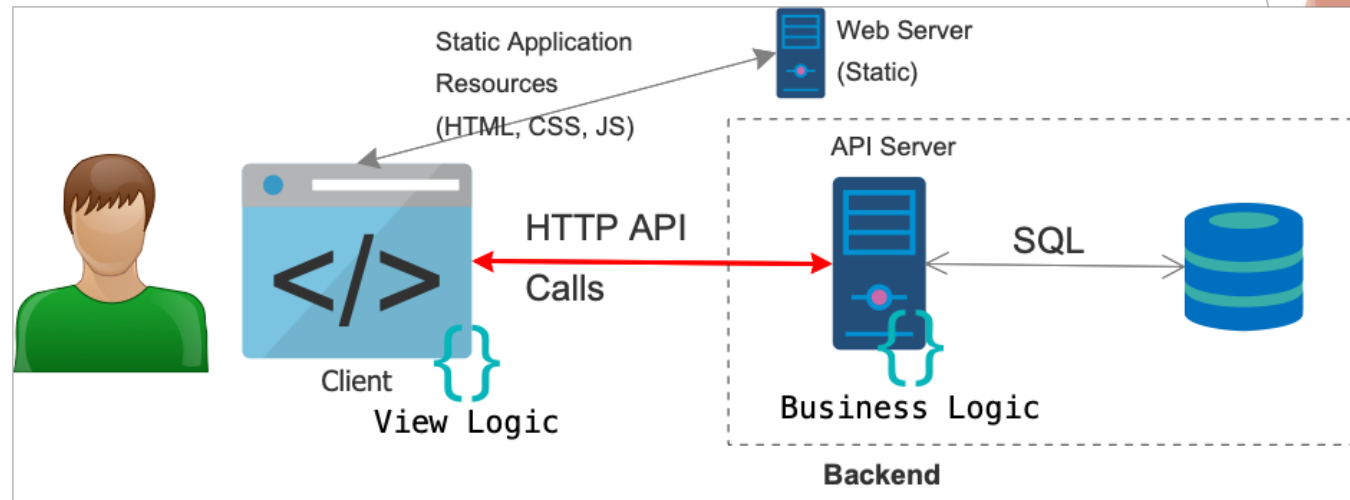
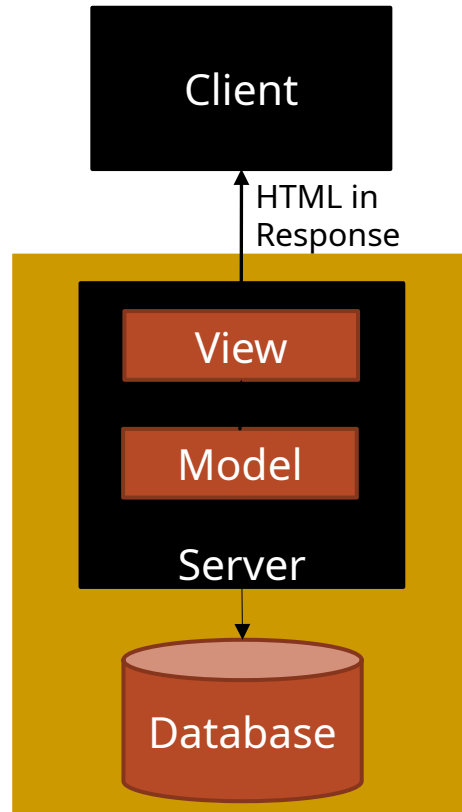


Model View Control (MVC)

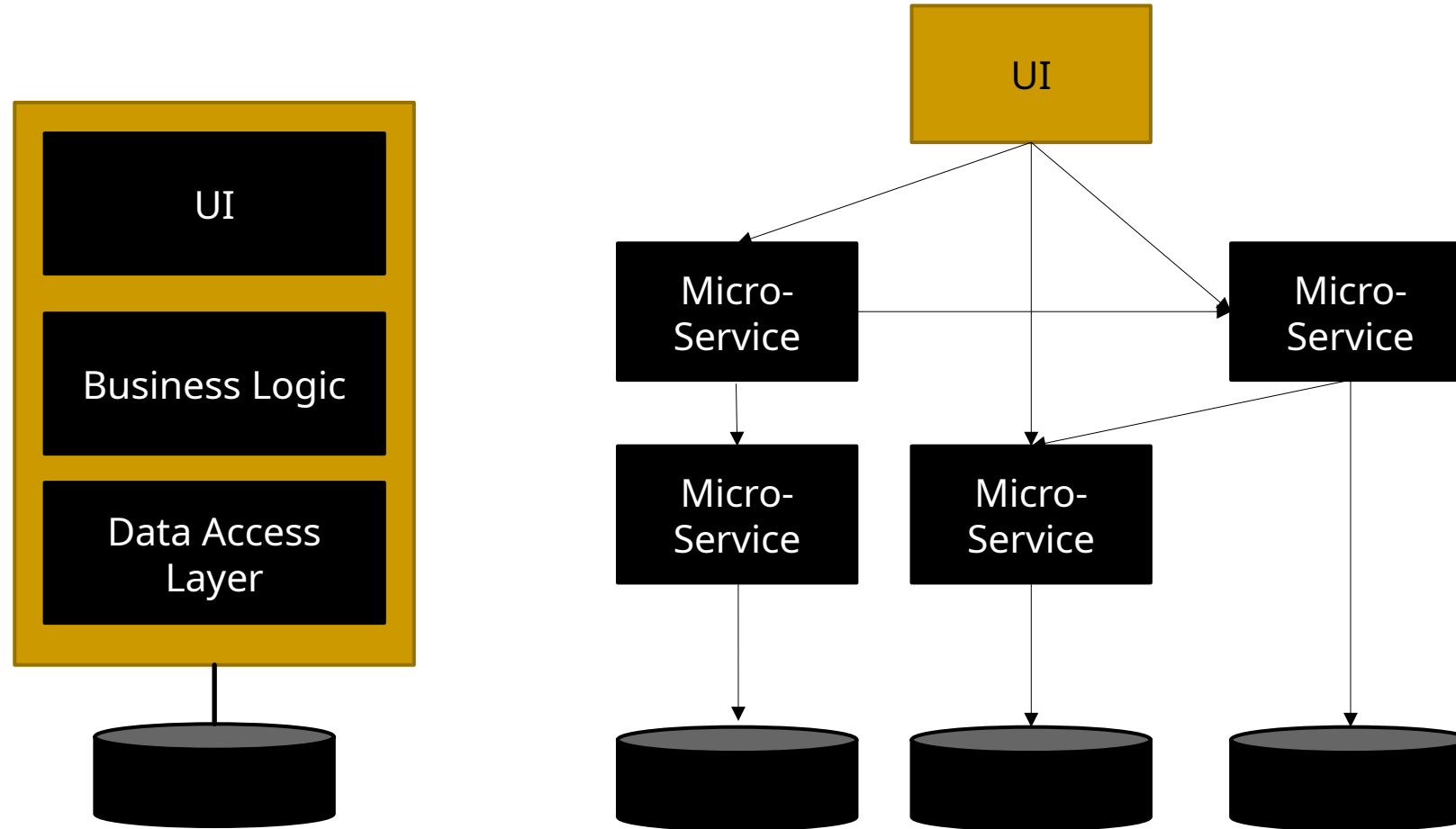


Traditional vs. Modern Web Applications

- ▶ Business logic and presentation logic performed **on the server**
- ▶ Technologies
 - ▶ Struts, Java Server Faces, etc
- ▶ View logic is implemented on client side
- ▶ Business data is loaded from a backend application via API
- ▶ Technologies (frontend)
 - ▶ Angular, Vue, React



Monolithic vs Microservice architecture

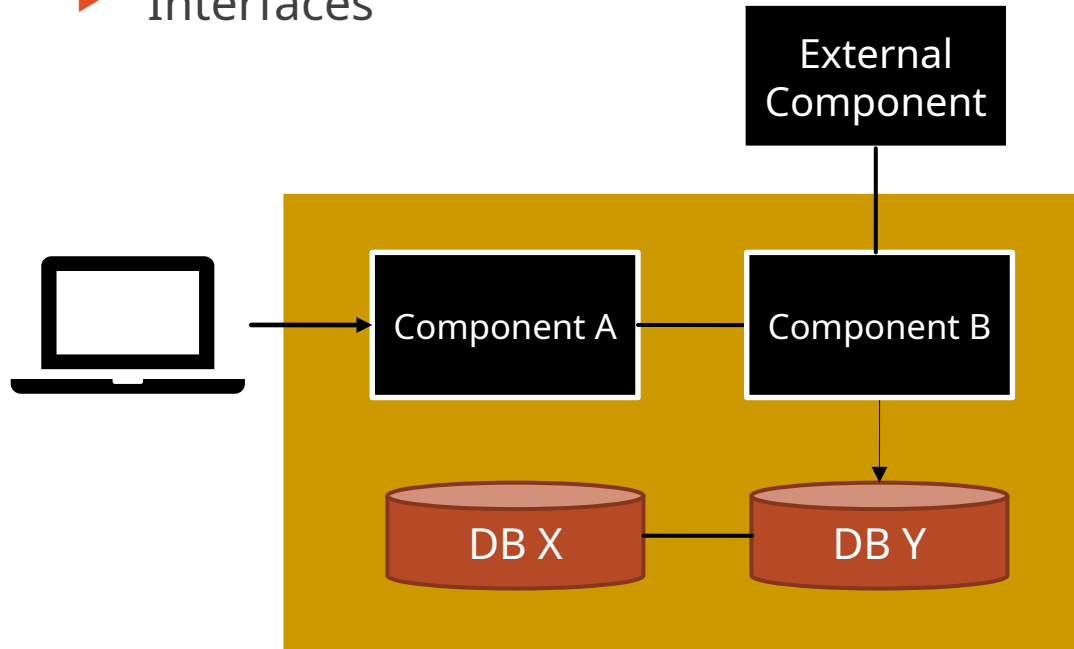


Interfaces (API)

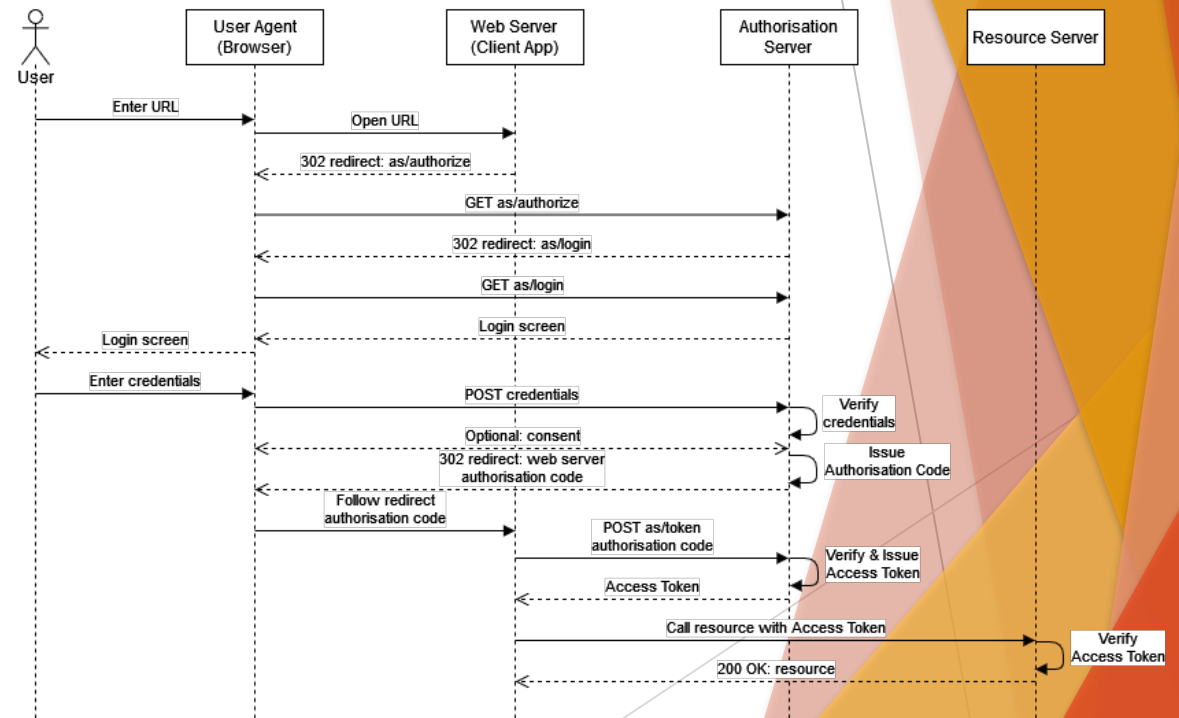
- ▶ An interface is a shared boundary between two or more components to exchange information or expose functionality
- ▶ An interface usually defines
 - ▶ **Data types**
 - ▶ **Structures**
 - ▶ **Operations**
 - ▶ **Protocols**
- ▶ An interface does not explicitly define implementation details

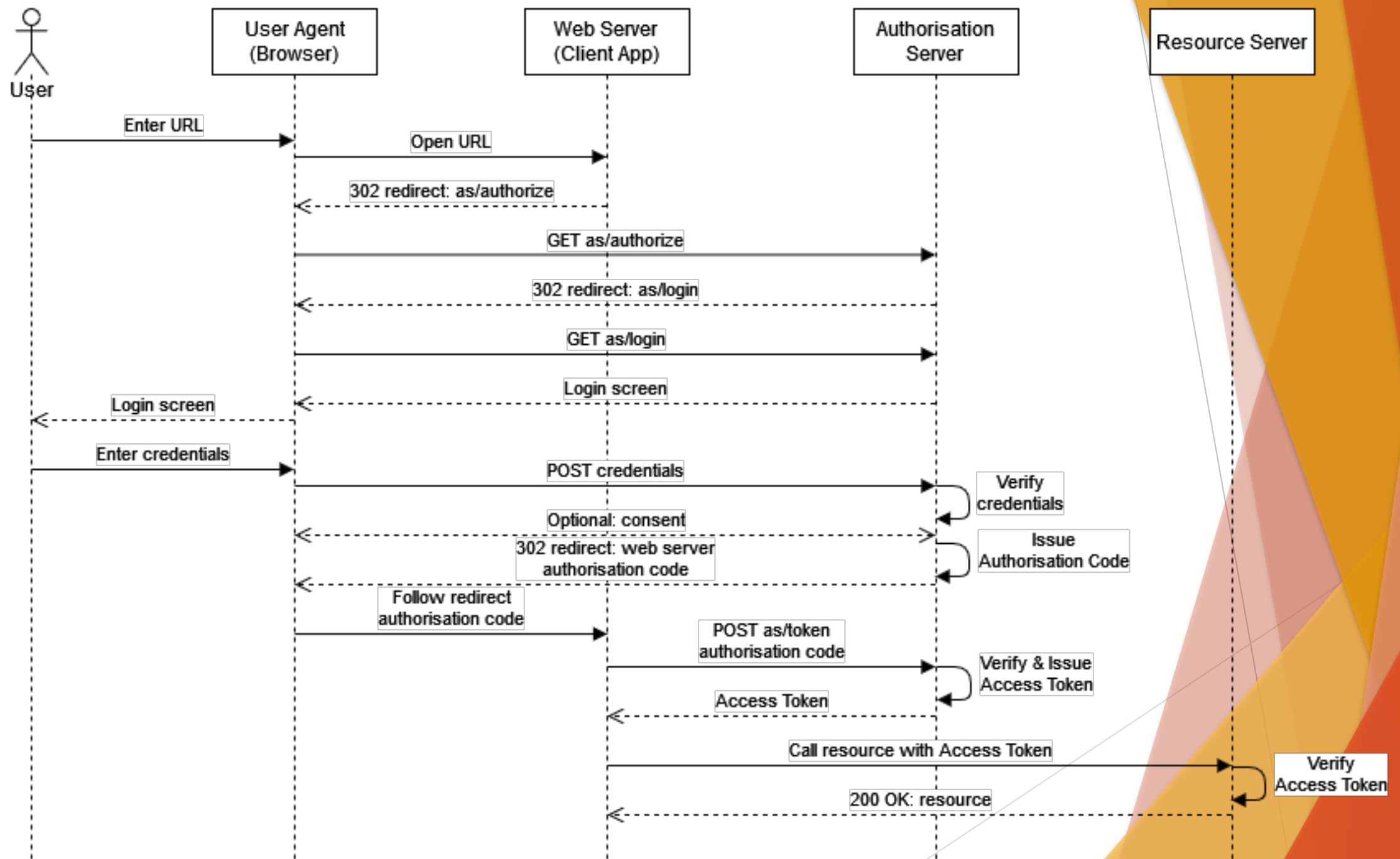
System Architecture vs. Sequence Diagram

- ▶ Component view
- ▶ Shows high-level components of a system
- ▶ Interfaces

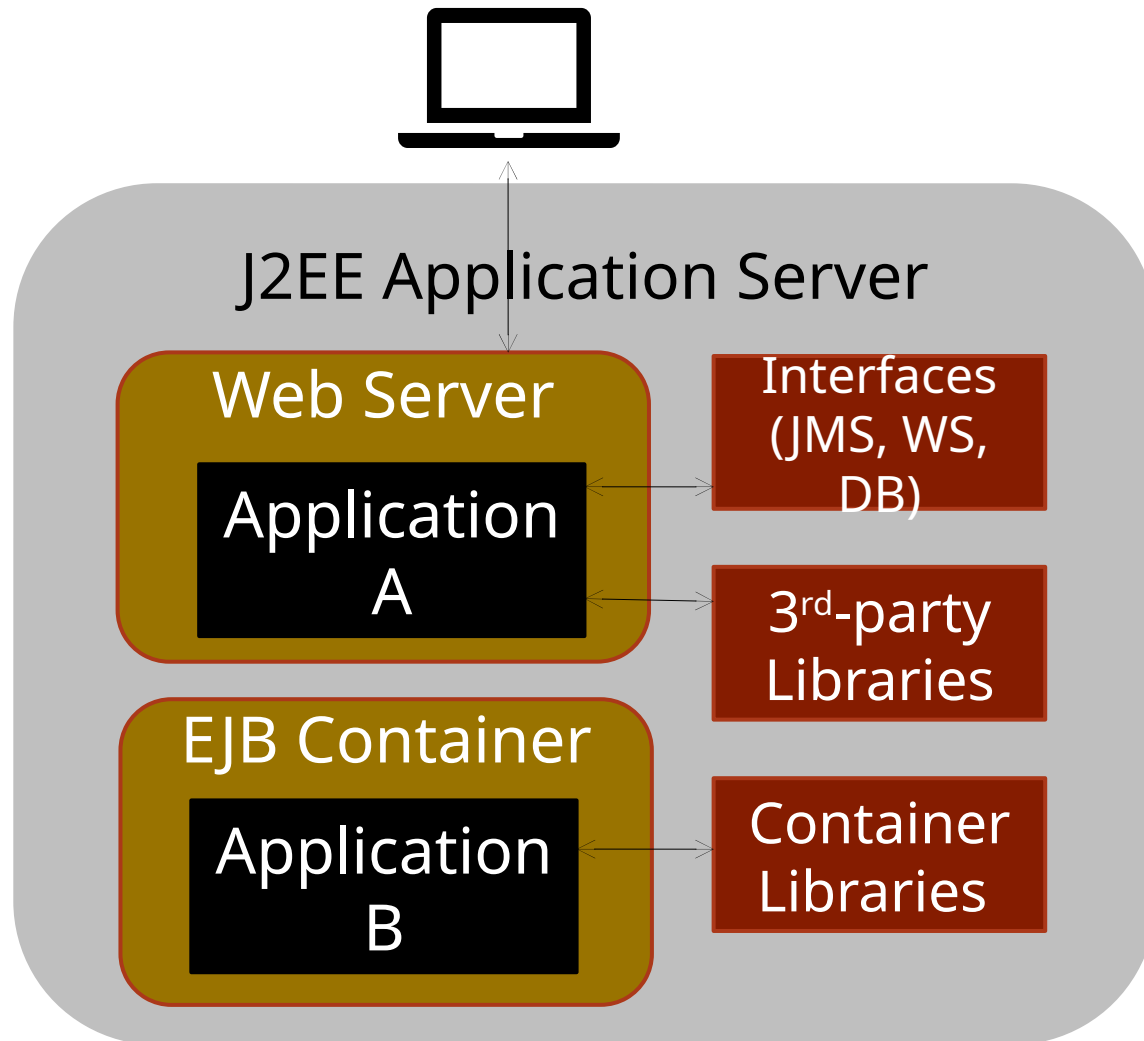


- ▶ Process flow of a use case





Application Server



Exercise: Architecture of a Web Application

1. Design a web application with multiple users
2. Draw the system architecture (block) diagram
3. Define a relevant use case and sketch the sequence diagram
4. Security considerations

Secure by Design

Secure designs lay the foundation of secure software

Security begins with the design

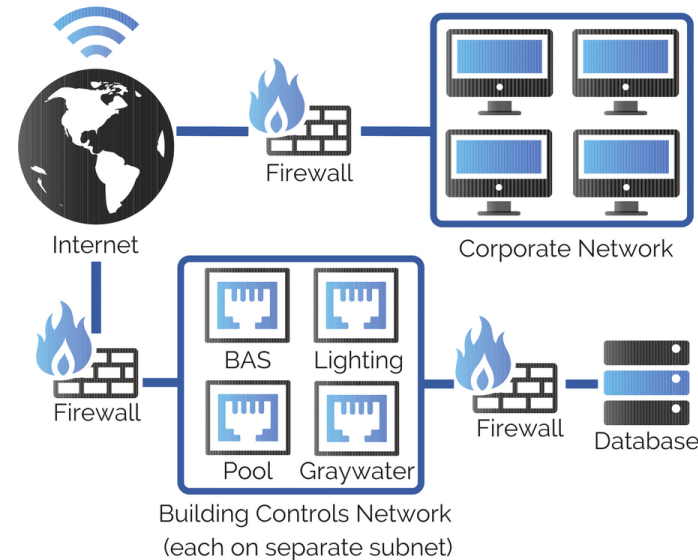
- Not only an implementation problem
- Easiest to address security

Neglect can lead to vulnerabilities

- Hardcoded credentials
- Missing input validation
- No data encryption
- ...

Secure design is not just cost

- Builds trust
- Reduces risks and future costs



Principles



Least Privilege



Defense in Depth



Minimum Exposure



Keep it simple



Fail securely



Ensure traceability



Segmentation



Encrypt everywhere



Secure and Proven Algorithms and Patterns



Defensive Programming

Least Privilege means not granting unnecessary access

```
struct group_info init_groups = { .usage = Atomic_init(2) };
struct group_info *groups_alloc(int gidsetsize){
    struct group_info *group_info;
    int nblocks;
    int i;

    nblocks = (gidsetsize + NGROUPS_PER_BLOCK - 1) / NGROUPS_PER_BLOCK;
    /* Make sure we always allocate at least one indirect block pointer */
    nblocks = nblocks ? : 1;
    group_info = kmalloc(sizeof(*group_info) + nblocks*sizeof(gid_t *), GFP_USER);
    if
    gro
    gro
    ato

    if (gidsetsize <= NGROUPS_SMALL)
        group_info->blocks[0] = group_info->small_block;
    else {
        for (i = 0; i < nblocks; i++) {
            gid_t *b;
            b = (void *)__get_free_page(GFP_USER);
            if (!b)
                goto out_undo_partial_alloc;
        }
    }
}
```

ACCESS DENIED

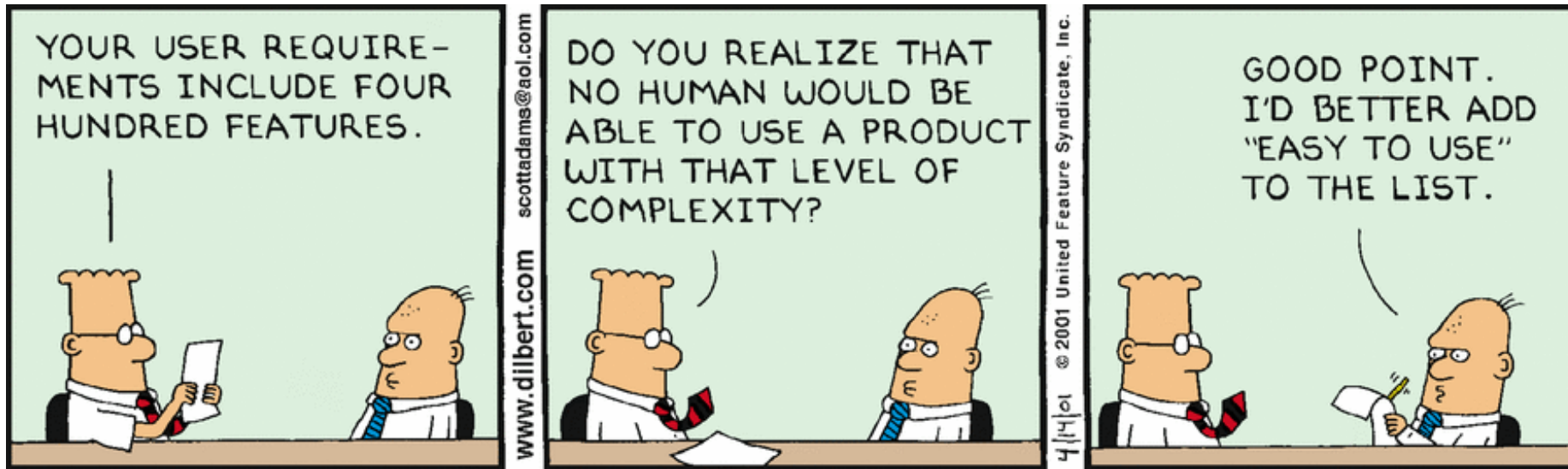
Defense in depth aka the castle approach



Minimising exposure means minimising risk



Keep things easy to understand



"Make everything as simple as possible, but not simpler." – Albert Einstein

Fail Securely

Definition

- Default to secure state in case of failure

Importance

- Prevent unauthorised access or leaks
- Maintain trust
- Ensure consistent security posture

Examples

- An application that logs users out during a crash
- A security door remains locked during power outage

Ensure Traceability

Leave an audit trail

- What resource was accessed, why and by whom
- What actions have been performed
- What functions have been called, etc.

Logs must provide information

- Security event alerts
- forensics



➔ ...but **do not** log confidential data!

Divide et impera

Segment applications

- Contain malicious activity
- “Lock it down”

Divide according to

- Functionality
- Security properties

Encryption



Do not reinvent the wheel

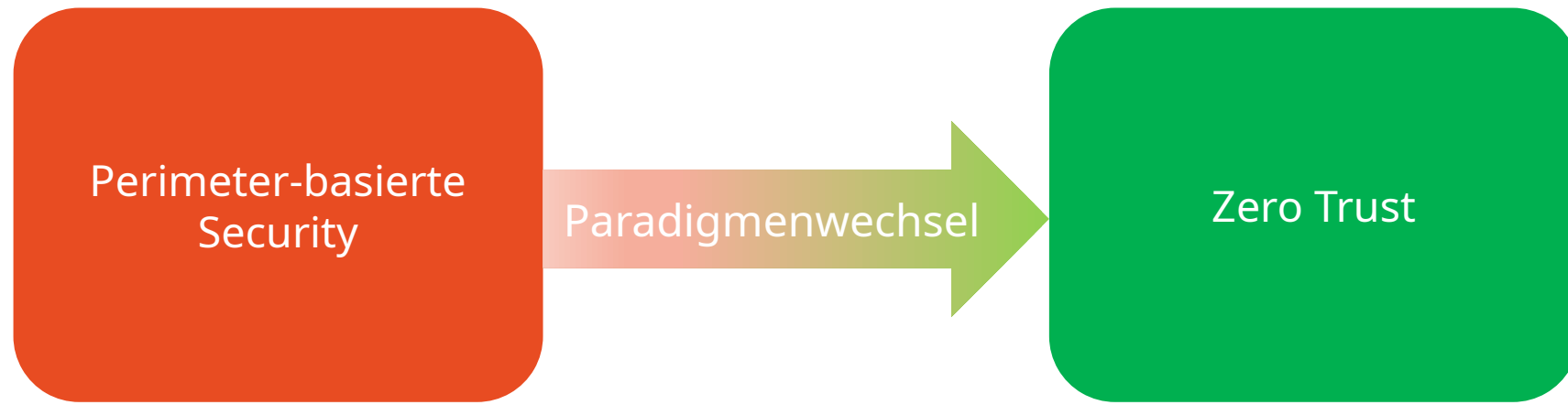


Defensive Programming



Zero Trust

Vertraue niemandem, verifiziere alles



Das Ziel ist, Angreifer von den Assets fern zu halten

Komplexität

- Mehr Geräte
- Mehr Benutzer
- Mehr Verbindungen

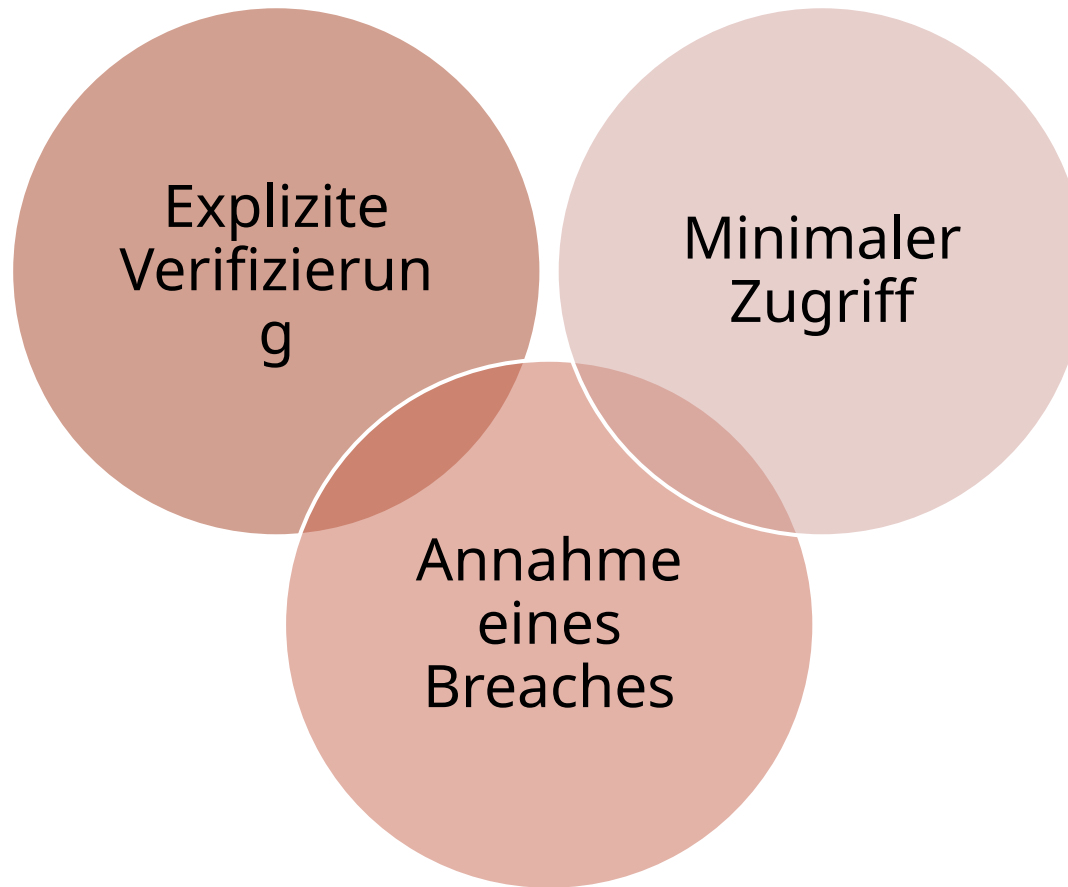
Veränderte IT Landschaft

- BYOD
- Home Office & Mobilität
- SaaS

Erhöhte Bedrohung

- Quantitativ & Qualitativ
- Identitätsangriffe
- Insider

Zero Trust ist weder Produkt noch Service



Excercises

- ▶ <https://overthewire.org/wargames/natas/>

